Computers and Chemical Engineering 32 (2008) 2903–2913

Contents lists available at ScienceDirect

Computers and Chemical Engineering

journal homepage: www.elsevier.com/locate/compchemeng

MPEC problem formulations and solution strategies with chemical engineering applications

B.T. Baumrucker[a], J.G. Renfro[b], L.T. Biegler[a,*]

[a] Chemical Engineering Department, Carnegie Mellon University, Pittsburgh, PA 15213, USA
[b] Honeywell International, 1250 West Sam Houston Parkway South, Houston, TX 77042, USA

ARTICLE INFO

Article history:
Received 16 June 2007
Received in revised form 25 February 2008
Accepted 26 February 2008
Available online 8 March 2008

Keywords:
Nonlinear programming
Process optimization
Complementarity
MPEC

ABSTRACT

With the development and widespread use of large-scale nonlinear programming (NLP) tools for process optimization, there has been an associated application of NLP formulations with complementarity constraints in order to represent discrete decisions. Also known as mathematical programs with equilibrium constraints (MPECs), these formulations can be used to model certain classes of discrete events and can be more efficient than a mixed integer formulation. However, MPEC formulations and solution strategies are not yet fully developed in process engineering. In this study, we discuss MPEC properties, including concepts of stationarity and linear independence that are essential for well-defined NLP formulations. Nonlinear programming based solution strategies for MPECs are then reviewed and examples of complementarity drawn from chemical engineering applications are presented to illustrate the effectiveness of these formulations.

© 2008 Elsevier Ltd. All rights reserved.

1. Introduction

Mathematical programming methods have proven extremely valuable for the design and operation of chemical processes. This has been made possible by more flexible modeling constructs, increased computing power, and a better fundamental understanding of solution algorithms and problem formulations. Related optimization models have a number of options to represent discrete decisions, as in contact and friction problems in computational mechanics, equilibrium relations in economics, and a wide variety of discrete events in process systems. Many of these situations naturally lend themselves to complementarity constraints (Ferris, 2005; Luo, Pang, & Ralph, 1996); however these mathematical programming formulations and related solution strategies are not yet fully developed in process engineering.

Complementarity is a relationship between variables where either one (or both) must be at its bound. Complementarities are particularly useful in optimization since they can be used to model disjunctions without the use of binary variables. Generalized disjunctive programming and mixed integer programming techniques are the most general approach to handle logical disjunctions. However, computational expense may be high for large systems with many discrete decisions, such as dynamic systems with switches at any point in time. With mixed integer formulations, the worst-case solution time grows exponentially with the number of discrete decisions. Complementarities offer an alternative for some classes of disjunctive problems and can be embedded within a standard nonlinear programming (NLP) solver to obtain fast local solutions, generally in polynomial time. However, the introduction of complements introduces an inherent nonconvexity as well as linear dependence of constraints.

In process optimization, complementarity models allow a number of options when dealing with disappearance of phases, flow reversal, safety valve operation, and other discrete events. Traditionally, sequential modular approaches to flowsheet simulation have handled discrete constraints through logic embedded in the software (IF statements). Depending on the value of computed variables, different sets of equations may be used for the simulation. However, this sequential modular approach has its limitations for complex recycles and information flow reversals. Moreover, its usage in an optimization framework has not been well-supported except for small problems. Instead, equation based flowsheet models are much better suited for use in an optimization framework and have the flexibility to handle complex specification changes and recycle modeling situations. However, equation based approaches must deal with discrete decisions in a different way, as embedded logic to change model equations is undesirable; it leads to nonsmooth relations and usually results in solver failures or convergence at nonphysical solution points of the model equations. In this context, complementarity models offer an efficient alternative that overcomes these problems.

Complementarities also arise naturally in the solution of mathematical programs with equilibrium constraints (MPECs) and

* Corresponding author. Tel.: +1 412 268 2232; fax: +1 412 268 7139.
E-mail address: lb01@andrew.cmu.edu (L.T. Biegler).

0098-1354/$ – see front matter © 2008 Elsevier Ltd. All rights reserved.
doi:10.1016/j.compchemeng.2008.02.010

bi-level optimization problems. An MPEC is a mathematical program constrained by a variational inequality, which takes the following general form:

$$\min \quad f(x, y) \tag{1a}$$

$$\text{s.t.} \quad (x, y) \in Z \tag{1b}$$

$$(\nu - y)^{\mathrm{T}} F(x, y) \geq 0, \quad y \in C(x),\ \forall \nu \in C(x) \tag{1c}$$

where Z is the feasible region for the upper level problem and $C(x)$ is the feasible region for the variational inequality. Variational inequalities arise in many domains and are generally referred to as equilibrium constraints. If the function $F(x, y)$ is a gradient map with respect to the second argument $F(x, y) = \nabla_y \theta(x, y)$ and $C(x)$ has a finite representation, the variational inequality is equivalent to an inner minimization problem (Luo et al., 1996). This gives rise to the following bilevel optimization problem:

$$\min_{x} \quad f(x, y) \tag{2a}$$

$$\text{s.t.} \quad (x, y) \in Z \tag{2b}$$

$$y = \arg\min_{\hat{y}} \{\theta(x, \hat{y}) : \hat{y} \in C(x)\} \tag{2c}$$

Bilevel optimization problems can be reformulated as a mathematical program with complementarity conditions (MPCCs) by writing the optimality conditions of the inner optimization problem as constraints on the outer problem. Conversely, any MPCC can be reformulated as an equivalent MPEC. Complementarity formulations of vapor–liquid phase equilibrium have been derived from the minimization of Gibbs free energy (Gopal & Biegler, 1999) and these complementarity conditions have been extended to the optimization of trayed distillation columns (Raghunathan, Diaz, & Biegler, 2004). Complementarity models have also been applied to parameter estimation of core flood and reservoir models (Kameswaran, Staus, & Biegler, 2005). Other uses of complementarities include optimization of metabolic flux networks (Raghunathan, Perez-Correa, Agosin, & Biegler, 2006), real time optimization (RTO) of chemical processes (Getting Started with ROMeo, 2004), and hybrid systems (Heemels, 1999).

An MPCC takes the following general form:

$$\min \quad f(x, y, z) \tag{3a}$$

$$\text{s.t.} \quad h(x, y, z) = 0 \tag{3b}$$

$$g(x, y, z) \geq 0 \tag{3c}$$

$$0 \leq x \perp y \geq 0 \tag{3d}$$

where $\perp$ is the complementarity operator enforcing at least one of the bounds to be active. (Note that we have classified the complementing variables as x, y, as well as the remaining variables, z.) This MPCC problem can be rewritten as the equivalent MPEC problem:

$$\min_{x,y,z} \quad f(x, y, z) \tag{4a}$$

$$\text{s.t.} \quad h(x, y, z) = 0 \tag{4b}$$

$$g(x, y, z) \geq 0, x \geq 0 \tag{4c}$$

$$y = \arg\{\min_{\hat{y}} x^{\mathrm{T}} \hat{y} \quad \text{s.t. } \hat{y} \geq 0\} \tag{4d}$$

The complementarity constraint (3d) implies the following:

$$x_i = 0 \quad \text{OR} \quad y_i = 0 \quad \forall i$$
$$x \geq 0, y \geq 0.$$

It can be seen that complementarity constraints implicitly model disjunctive behavior. Here the OR operator is inclusive as both variables may be zero. Alternatively the complementarity constraint may be written in several equivalent forms:

$$x^{\mathrm{T}} y = 0, \quad x \geq 0, y \geq 0 \tag{5}$$

$$x_i y_i = 0 \quad \forall i, \qquad x \geq 0, y \geq 0 \tag{6}$$

$$x_i y_i \leq 0 \quad \forall i, \qquad x \geq 0, y \geq 0 \tag{7}$$

These alternative forms are particularly useful when applying existing nonlinear programming (NLP) solution strategies to solve MPCCs.

Section 2 summarizes MPEC properties that support their reformulation as standard NLP problems. The NLP formulations are presented in Section 3. Following this, Section 4 describes commercially available software that automates these reformulation strategies; it then compares and evaluates these formulations along with standard NLP solvers. Section 5 then considers the modeling of discrete decisions with MPEC formulations and details several important chemical engineering applications of MPCCs. To demonstrate both the models and solution strategies, two case studies are considered in Section 6. First, a discretized optimal control problem demonstrates the scalability of MPCC problems. Second, we consider a detailed case study on the design of trayed distillation columns. Section 7 then concludes the paper and offers directions for future work.

2. Properties of MPECs

Despite the advantages that complementarities offer, their use is tempered by their lack of NLP regularity properties. In particular, at all feasible points MPCCs violate the linear independence constraint qualification (LICQ), which requires the gradients of the active constraints to be linearly independent. This systemic violation can be seen in the following small example:

$$\min \quad f(x, y) \tag{8a}$$

$$\text{s.t.} \quad 0 \leq x \perp y \geq 0 \tag{8b}$$

This problem may be rewritten in the following equivalent form:

$$\min \quad f(x, y) \tag{9a}$$

$$\text{s.t.} \quad g_1 : x \geq 0 \tag{9b}$$

$$g_2 : y \geq 0 \tag{9c}$$

$$g_3 : x^{\mathrm{T}} y \leq 0 \tag{9d}$$

Assuming that $x = 0$ at the solution, then the active set $c(x, y) = 0$ consists of g_1 and g_3 where

$$\nabla g_1 = [1 \quad 0]^{\mathrm{T}} \tag{10}$$

$$\nabla g_3 = [y \quad 0]^{\mathrm{T}} \tag{11}$$

$$\nabla c^{\mathrm{T}} = \begin{bmatrix} 1 & 0 \\ y & 0 \end{bmatrix} \Rightarrow \text{linearly dependent} \tag{12}$$

Failure of LICQ is noteworthy as uniqueness of the constraint multipliers holds if and only if LICQ is satisfied. A weaker condition is the Mangasarian–Fromovitz constraint qualification (MFCQ), which requires linear independence of the equality constraints and the existence of a search direction that lies both in the null space of the equality constraint gradients and points into the interior of the region defined by the linearized active inequality constraints. MFCQ is a necessary and sufficient condition for boundedness of the multipliers. However, MPCCs violate MFCQ at all feasible points as well. Consequently, multipliers of the MPCC (3) will be nonunique and unbounded.

Because these constraint qualifications do not hold, it is not surprising that finding an MPCC solution may be difficult. In order to classify this solution, we introduce the definition of a *B*-stationary point for an MPCC. A point is a *B-stationary point* if it is feasible to the MPCC and $d = 0$ is the solution to the following linear program with equilibrium constraints (LPEC) (Luo et al., 1996; Ralph & Wright, 2004; Scheel & Scholtes, 2000):

$$\min_d \quad \nabla f(w^*)^{\mathrm{T}} d \tag{13a}$$

$$\text{s.t.} \quad g(w^*) + \nabla g(w^*)^{\mathrm{T}} d \geq 0 \tag{13b}$$

$$h(w^*) + \nabla h(w^*)^{\mathrm{T}} d = 0 \tag{13c}$$

$$0 \leq x^* + d_x \perp y^* + d_y \geq 0 \tag{13d}$$

where

$$w^* = (x^*, y^*, z^*)$$

Verification of this condition may require the solution of 2^m linear programs where m is the cardinality of the biactive set $I_X \cap I_Y$ where

$$I_X = \{i : x_i^* = 0\}$$
$$I_Y = \{i : y_i^* = 0\}$$

A stronger, but more tractable form of stationarity for the MPCC problem is *strong stationarity*. A point is strongly stationary if it is feasible for the MPCC and $d = 0$ solves the following relaxed problem:

$$\min_d \quad \nabla f(w^*)^{\mathrm{T}} d \tag{14a}$$

$$\text{s.t.} \quad g(w^*) + \nabla g(w^*)^{\mathrm{T}} d \geq 0 \tag{14b}$$

$$h(w^*) + \nabla h(w^*)^{\mathrm{T}} d = 0 \tag{14c}$$

$$d_{x,i} = 0, \quad i \in I_X \setminus I_Y \tag{14d}$$

$$d_{y,i} = 0, \quad i \in I_Y \setminus I_X \tag{14e}$$

$$d_{x,i} \geq 0, \quad i \in I_X \cap I_Y \tag{14f}$$

$$d_{y,i} \geq 0, \quad i \in I_X \cap I_Y \tag{14g}$$

Strong stationarity implies no feasible descent direction for (14) and is equivalent to B-stationarity if the biactive set $I_X \cap I_Y$ is empty, or if the MPEC-LICQ property holds (Anitescu, Tseng, & Wright, 2007). MPEC-LICQ requires that the following set of vectors be linearly independent:

$$\{\nabla g_i(w^*) | i \in I_g\} \cup \{\nabla h(w^*)\} \cup \{\nabla x_i | i \in I_X\} \cup \{\nabla y_i | i \in I_Y\} \tag{15}$$

where

$$I_g = \{i : g_i(w^*) = 0\}.$$

MPEC-LICQ implies that the multipliers of (14) are bounded and unique. It has been shown that MPEC-LICQ is a generic property of MPCCs and of bilevel problems formulated as MPCC (Scholtes & Stöhr, 2001) and that this property can be expected to hold "almost everywhere". That is, if MPEC-LICQ is violated for a particular MPEC, it can always be satisfied by small perturbations of that problem. Satisfaction of MPEC-LICQ leads to the following result:

Theorem 1 ((Anitescu et al., 2007; Leyffer, Lopez-Calva, & Nocedal, 2006; Scheel & Scholtes, 2000)). *If w^* is a solution to the MPCC* (3) *and MPEC-LICQ holds at w^*, then w^* is strongly stationary.*

With this property, convergence to a strongly stationary point is much easier (and required for many NLP algorithms), since only one set of MPEC multipliers is needed to verify optimality and, consequently, this property allows the reformulation of (3) to a number of equivalent nonlinear programs. In the past, special MPEC solvers were required (Ferris & Pang, 1997; Ferris, 2005; Luo et al., 1996) to handle these difficulties. With this property, MPECs can now be addressed directly through reformulation and direct solution as NLPs. These NLP reformulations are covered in the next section.

Finally, a related (implied by MPEC-LICQ, but weaker) constraint qualification is MPEC-MFCQ, which requires that there exists a nonzero vector $d \in \mathbb{R}^n$ such that:

$$d_{x,i} = 0, \quad i \in I_X \setminus I_Y \tag{16a}$$

$$d_{y,i} = 0, \quad i \in I_Y \setminus I_X \tag{16b}$$

$$\nabla h(w^*)^{\mathrm{T}} d = 0 \tag{16c}$$

$$\nabla g_i(w^*)^{\mathrm{T}} d > 0 \quad i \in I_g \tag{16d}$$

$$d_{x,i} > 0, \qquad d_{y,i} > 0 \quad i \in I_X \cup I_Y \tag{16e}$$

$$\{\nabla h(w^*)\} \cup \{\nabla x_i | i \in I_X\} \cup \{\nabla y_i | i \in I_Y\} \text{ are linearly independent} \tag{16f}$$

MPEC-MFCQ implies that the multipliers of (14) will be bounded.

3. Solution strategies

If no reformulation of an MPCC is made and the complementarity is represented by (5)–(7), the constraint multipliers will be nonunique and unbounded, and dependent active constraints exist at every feasible point. However, some active set NLP strategies may be able to overcome this difficulty as they were developed to handle degenerate NLP formulations (Burke & Han, 1989) and increase robustness of active set methods. In particular, encouraging results have been reported with the Filter-SQP algorithm (Fletcher & Leyffer, 2004). Moreover, elastic mode SQP algorithms have finite termination and global convergence properties when solving MPCCs (Anitescu et al., 2007). On the other hand, an active set algorithm's performance is still affected by combinatorial complexity as the problem size increases.

The following MPCC reformulations have been analyzed in Anitescu et al. (2007), Hu and Ralph (2004), Leyffer et al. (2006), Raghunathan and Biegler (2005), and Ralph and Wright (2004) and allow standard NLP tools to be applied.

$$\text{Reg}(\epsilon): \quad \min \quad f(w) \tag{17a}$$

$$\text{s.t.} \quad h(w) = 0 \tag{17b}$$

$$g(w) \geq 0 \tag{17c}$$

$$x, y \geq 0 \tag{17d}$$

$$x_i y_i \leq \epsilon \quad \forall i \tag{17e}$$

$$\text{RegComp}(\epsilon): \quad \min \quad f(w) \tag{18a}$$

$$\text{s.t.} \quad h(w) = 0 \tag{18b}$$

$$g(w) \geq 0 \tag{18c}$$

$$x, y \geq 0 \tag{18d}$$

$$x^{\mathrm{T}} y \leq \epsilon \tag{18e}$$

$$\text{RegEq}(\epsilon): \quad \min \quad f(w) \tag{19a}$$

$$\text{s.t.} \quad h(w) = 0 \tag{19b}$$

$$g(w) \geq 0 \tag{19c}$$

$$x, y \geq 0 \tag{19d}$$

$$x_i y_i = \epsilon \quad \forall i \tag{19e}$$

$$\text{PF}(\rho): \quad \min \quad f(w) + \rho x^{\mathrm{T}} y \tag{20a}$$

$$\text{s.t.} \quad h(w) = 0 \tag{20b}$$

$$g(w) \geq 0 \tag{20c}$$

$$x, y \geq 0 \tag{20d}$$

Complementarity conditions may be relaxed and the problem reformulated as in the Reg(ϵ), RegComp(ϵ), or RegEq(ϵ) with a positive relaxation parameter ϵ. The solution of the MPCC, w^*, can be obtained by solving a series of relaxed solutions, $w(\epsilon)$, as ϵ approaches zero. These solution strategies are generally attracted to strongly stationary points, but with some exceptions (see Anitescu et al., 2007). Under assumptions of MPEC-MFCQ and sufficient second order MPEC conditions (Ralph & Wright, 2004), Reg(ϵ) and RegComp(ϵ) exhibit local convergence of $\|w(\epsilon) - w^*\| \leq O(\epsilon^{1/2})$. (These properties can be strengthened to $O(\epsilon)$ if we can assume strict complementarity of the bound multipliers for $i \in I_X \cup I_Y$.) Reg(ϵ) has been implemented and tested in a version of IPOPT called IPOPT-C (Raghunathan & Biegler, 2005). On the other hand, RegEq(ϵ) will exhibit slower convergence with $\|w(\epsilon) - w^*\| \leq O(\epsilon^{1/4})$ under the same assumptions.

In a related development, nonlinear complementarity problem (NCP) functions and smoothing functions have also been used extensively to solve MPCCs (Chen & Fukushima, 2004; Gopal & Biegler, 1999; Lang & Biegler, 2002; Stein, Oldenburg, & Marquardt, 2004). NCP functions replace the complementarity condition in the problem with an equivalent nonlinear equation. One widely studied NCP function is the Fischer–Burmeister function:

$$\phi(x, y) = x + y - \sqrt{x^2 + y^2} \tag{21}$$

As this function is nondifferentiable at $x = y = 0$, it is usually smoothed to the form:

$$\phi(x, y) = x + y - \sqrt{x^2 + y^2 + \epsilon} \tag{22}$$

for some small $\epsilon > 0$. The solution to the original problem is then recovered by solving a series of problems as ϵ approaches zero. An equivalence can be made between this method and RegEq(ϵ). Accordingly, the problem will converge at the same slow convergence rate.

In contrast to the regularized formulations, we also consider the exact ℓ_1 penalization shown in PF(ρ). Here the complementarity can be moved from the constraints to the objective function and the resulting problem is solved for a particular value of ρ. If $\rho \geq \rho_c$, where ρ_c is the critical value of the penalty parameter, then the complementarity constraints will be satisfied at the solution. Similarly, Anitescu et al. (2007) consider a related "elastic mode" formulation, where artificial variables are introduced to relax the constraints in PF(ρ) and an additional ℓ_∞ constraint penalty term is added. In both cases the resulting NLP formulation has the following properties:

Theorem 2 ((Ralph & Wright, 2004)). *If w^* is a strongly stationary point for the MPCC* (3), *then for all ρ sufficiently large w^* is a stationary point for* PF(ρ).

Theorem 3 ((Anitescu et al., 2007)). *If w^* is a solution to* PF(ρ) *and w^* is feasible for the MPCC* (3), *then w^* is a strongly stationary solution to* (3).

Theorem 2 indicates that the solution strategy is attracted to strongly stationary points for sufficiently large values of ρ. It is not known beforehand how large ρ must be. If the initial value of ρ is too small a series of problems with increasing ρ values may need to be solved. Theorem 3 indicates that a local solution of PF(ρ) is a solution to the MPCC if it is also feasible to the MPCC. This condition is essential as there is no guarantee that PF(ρ) will not get stuck at a local solution that does not satisfy the complementarity, as observed by Hu and Ralph (2004). With a finite value of $\rho > \rho_c$ this ℓ_1 penalization is exact in the sense that all strongly stationary points of the MPCC are local minimizers to PF(ρ). PF(ρ) allows any general nonlinear programming solver to be used to solve a complementarity problem. Provided that the penalty parameter is large enough, the MPCC may then be solved as a single problem, instead of a series of problems. Finally, in order to provide greater numerical efficiency, it has been suggested to dynamically change the penalty parameter during the course of the optimization for both active set and interior point optimization algorithms (Anitescu et al., 2007; Leyffer et al., 2006). This modification was recently introduced into the solver KNITRO (Lopez-Calva, 2005).

Finally, there is a strong interaction between the MPCC reformulation and the applied NLP solver. In particular, as noted in Leyffer et al. (2006), if a barrier NLP method (like KNITRO or IPOPT) is applied to PF(ρ), then there is a one-to-one correspondence of intermediate solutions of PF(ρ), Reg(ϵ) and RegComp(ϵ). This equivalence can be seen by comparing the penalty parameter ρ to the multiplier associated with the relaxed complementarity constraints in Reg(ϵ) and RegComp(ϵ). As a result, for a corresponding set of parameters ρ and ϵ we would expect a barrier NLP method to exhibit similar performance with either PF(ρ), Reg(ϵ) or RegComp(ϵ).

4. MPECLib Results

The formulations of the previous section provide a straightforward way to represent and address any MPCC through nonlinear programming. An important component of this task is the efficiency and reliability of the NLP solver. In this section we compare and evaluate each of these formulations through a library test set provided with the GAMS modeling environment (GAMS, 2004). For this evaluation we consider MPECLib, a large collection of test problems, and apply an automatic reformulation of MPECs provided by the NLPEC meta-solver in GAMS.

4.1. Automatic reformulation of complementarity

MPCC models written in the GAMS modeling language may be automatically reformulated as an NLP using several of the previously described complementarity solution techniques. This can be accomplished with the use of the NLPEC meta-solver (GAMS, 2004). When called, NLPEC reformulates the complementarity constraints of a MPCC model with a user specified reformulation, including (17)–(20). NLPEC will then call a user-specified NLP solver to solve the reformulated model. The final results from the NLP solver are then translated back into the original MPCC model. NLPEC verifies that the complementarities are satisfied in the resulting solution. This is particularly important when using the PF(ρ) formulation described above, as it may not converge to a stationary point of the MPCC if the penalty parameter is too small.

4.2. Comparison of solution strategies on MPECLib

MPECLib is a set of MPEC test problems maintained by Dirkse (MPEC Library). The test set currently consists of 92 problems, which include both small scale models from the literature and several large industrial models. Some of the problems in the library include initial points to start the optimization. Those that do not were started from the GAMS default initial point.

The performance and robustness of several NLP reformulations including PF(ρ) with $\rho = 10$, Reg(ϵ) with $\epsilon = 10^{-8}$, and NCP formulation $x^T y = 0, x, y \geq 0$ were compared on the MPECLib test set. CONOPT (Version 3.14) and IPOPT (Version 3.2.3) were used to the solve the resulting NLP problems. All reformulations were performed automatically by the NLPEC solver described earlier. Also included in the comparison is the IPOPT-C solver, which is capable of processing the complementarity constraints without reformulation. The CONOPT and IPOPT runs were performed entirely within the GAMS environment. However, IPOPT-C is not currently linked to

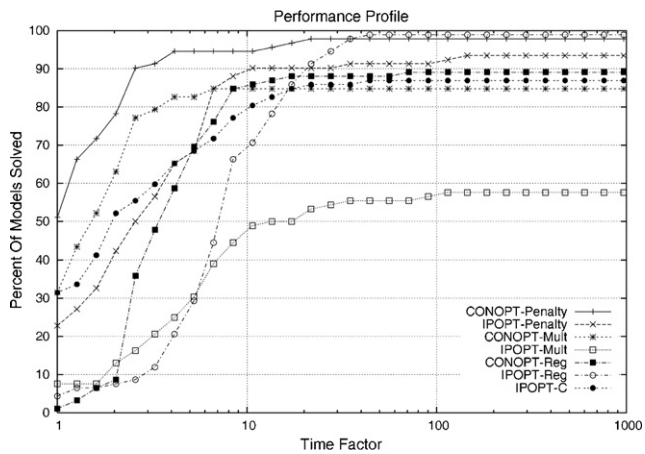

Fig. 1. Performance profiles of several reformulations on all of the 92 MPECLib test problems.

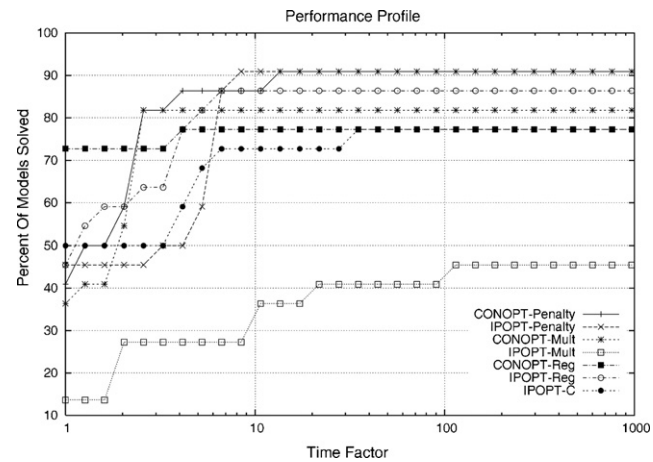

Fig. 2. Performance profiles of several reformulations on 22 select test problems.

GAMS. In order to perform the tests with IPOPT-C, the GAMS models were instead converted to the AMPL modeling language format. The problems were then run using AMPL and IPOPT-C.

The results are presented in a Dolan–Morè plot. All results were obtained on a Intel Pentium 4, 1.8 GHz computer with 992 MB of RAM. This plot portrays both robustness and relative performance of a set of solvers on a given problem set. Here we define a performance ratio calculated as

$$\eta_{p,s} = \frac{t_{p,s}}{\min\{t_{p,s} : 1 \le s \le n_s\}} \tag{23}$$

where n_s is the number of solvers, which are run over the n_p problems in set P, and where $t_{p,s}$ is the solution time for solver s on problem p. For all solvers s that fail to solve problem p, we set the performance ratio to an arbitrarily large number. The profiles are then defined by

$$p_s(\tau) = \frac{1}{n_p}\text{size}\{p \in P : \eta_{p,s} \le \tau\} \tag{24}$$

and $p_s(\tau)$ is the percentage of problems solved within $\tau \ge \eta_{p,s}$ times the best performance. This analysis can be performed automatically using the PAVER server (Mittelmann & Pruessner, 2006).

Fig. 1 shows the performance plot of different reformulations and solvers tested on all 92 problems of the test set. Since the problems are inherently nonconvex, multiple local solutions can be expected for each problem. Because of this, not all of the different reformulation and solver combinations converged to the same local solutions. This makes a direct comparison of performance difficult as the solvers often converge to different solutions. On the other hand, this figure is useful to demonstrate the robustness of the methods, as seen by $\lim_{\tau\to\infty} p_s(\tau)$. For all but one solution method, the reformulation and solver combination was able to solve at least 84% of all of the test problems. The most reliable solvers were IPOPT-Reg(ϵ) (99%), CONOPT-Penalty (98%) and IPOPT-Penalty (94%). In contrast, IPOPT with the NCP formulation (listed as IPOPT-Mult on the figure) was only able to solve 57% of all of the problems. This is not entirely unexpected as IPOPT is an interior point algorithm and the NCP formulation provides no strict interior to the NLP problem.

Fig. 2 shows the same reformulations and solvers tested as before except on only 22 of the 92 test problems. These 22 problems all converged to the same solution for all reformulations and solvers when they converged. This allows for a fair comparison of the solvers' performance. Table 1 displays these 22 test problems and their respective objective function values.

Table 1
Objective function values of the 22 MPECLib test problems that converged to same solutions

Problem name	Objective function value	Constraints	Variables	Complementarities
bard2	−6600	10	13	8
bard3	−12.67872	6	7	4
bartruss3_0	3.54545E−07	29	36	26
bartruss3_1	3.54545E−07	29	36	11
bartruss3_2	10166.57	29	36	6
bartruss3_3	3E−07	27	34	26
bartruss3_4	3E−07	27	34	11
bartruss3_5	10166.57	27	34	6
desilva	−1	5	7	4
ex9_1_4m	−61	5	6	4
findb10s	2.02139E−07	203	198	176
fjq1	3.207699	7	8	6
gauvin	20	3	4	2
kehoe1	3.6345595	11	11	5
outrata31	2.882722	5	6	4
outrata33	2.888119	5	6	4
outrata34	5.7892185	5	6	4
qvi	7.67061E−19	3	5	3
three	2.58284E−20	4	3	1
tinque_dhs2	N/A	4834	4805	3552
tinque_sws3	12467.56	5699	5671	4480
tollmpec	−20.82589	2377	2380	2376

A few observations can be made on these results. CONOPT-Reg(ϵ) performed the fastest on 73% of the problems but it turns out to solve only 76% of them. Instead, CONOPT-Penalty and IPOPT-Penalty (PF(ρ)) are the next best in performance and they also prove to the most robust, solving over 90% of the problems. These turn out to be the best all-around methods.

For the results in Fig. 2, the CONOPT formulations take advantage of the active set strategy and particularly the detection and removal of degenerate constraints. As a result, the CONOPT-Mult (with the NCP reformulation) performs well. Because the test problems are not large, CONOPT provides the best performance on this test set.

On the other hand, the IPOPT formulations (IPOPT-Penalty, IPOPT-Reg, IPOPT-C) follow similar trends within Fig. 2, with the penalty formulation as the most robust. This can be explained by the similarities among these methods, as analyzed in Leyffer et al. (2006). Finally, IPOPT-Mult (with the NCP reformulation) is the worst performer as it cannot remove dependent constraints and therefore suffers most from the inherent degeneracies in MPCCs.

5. Process engineering examples of MPCCs

Complementarity applications in chemical engineering are widespread for modeling discrete decisions. Their use for modeling and optimizing split range controllers and tiered splitters has been known for some time (Gallun, Luecke, Scott, & Morshedi, 1992). However, with a better understanding of NLP-based solution strategies for MPCCs, it is essential to consider the formulation of well-posed complementarity models. In this section, we consider a systematic MPCC modeling strategy and apply it to a number of widely applied process examples.

To develop these concepts, we again caution that complementarity formulations are nonconvex problems and may lead to multiple local solutions. Consequently, with the application of standard NLP solvers, the user will need to be satisfied with local solutions, unless additional problem-specific information can be applied. Despite the nonconvexity in all MPCC formulations, it is clear that poorly posed MPCC formulations also need to be avoided. For instance, the feasible region for

$$0 \le y \perp (1-y) \ge 0 \tag{25}$$

consists of only two isolated points, and the disjoint feasible region associated with this example often leads to convergence failures, unless the model is initialized close to a solution with either $y = 0$ or $y = 1$. Moreover, without careful consideration, it is not difficult to create MPCCs with similar difficulties.

Because the MPCC can be derived from an associated MPEC, we consider the formulation in (2). To avoid disjoint regions, we require that both $\theta(x, y)$ and $C(x)$ be convex in y for all $(x, y) \in Z$. This leads to a well-defined solution for the inner problem. We use this observation to develop the following guidelines for the examples in this study:

- Start with the inner level problem:

$$\min_y \varphi(x)y \quad \text{s.t.} \quad y_a \le y \le y_b \tag{26}$$

  where $\varphi(x)$ is a switching function for the discrete decision.
- For (26) it is especially important that the upper level constraints in (2), i.e., $(x, y) \in Z$, not limit the selection of any value of $y \in C(x)$.
- The resulting MPEC is then converted to an MPCC. For instance, applying the KKT conditions to (26) we obtain the complementarities

$$\varphi(x) - s_a + s_b = 0 \tag{27a}$$

$$0 \le (y - y_a) \perp s_a \ge 0 \tag{27b}$$

$$0 \le (y_b - y) \perp s_b \ge 0 \tag{27c}$$

- The relations in (27) can be further simplified through variable elimination and application of the complementarity conditions.
- Finally, the resulting complementarity conditions are incorporated within NLPs using the formulations described in Section 3.

Note that in the absence of upper level constraints, the solution of (27) allows y to take any value in $[y_a, y_b]$ when $\varphi(x) = 0$. This is necessary to avoid disjoint feasible regions for the resulting MPCC. As a result, limitations on y from upper level constraints should be avoided, and this is frequently a problem specific task. For instance, complementarity formulations should not be used to model logical disjunctions such as an `EXCLUSIVE_OR` (as in (25)) because they lead to disjoint regions for y. In general, logical disjunctions such as the `INCLUSIVE_OR` can be modeled successfully with MPCCs.

In the remainder of this section, we apply these guidelines to develop complementarity models that arise in process applications. A number of these can also be found in Raghunathan (2004).

5.1. Commonly used functions

The following nonsmooth functions can be represented by complementarity formulations.

- The *signum* or *sign* operation $y = \text{sgn}(x)$ can be represented by a complementarity formulation:

$$\min \quad -yx \tag{28a}$$

$$\text{s.t.} \quad -1 \le y \le 1 \tag{28b}$$

  and the corresponding complementarity relations can be written as

$$x = s_b - s_a \tag{29a}$$

$$0 \le s_a \perp (y+1) \ge 0 \tag{29b}$$

$$0 \le s_b \perp (1-y) \ge 0 \tag{29c}$$

- The absolute value operator $z = |f(x)|$ can be rewritten as

$$z = f(x)y \tag{30}$$

$$y = arg\{\min_{\hat{y}} -f(x)\hat{y} \quad \text{s.t.} \quad -1 \le \hat{y} \le 1\} \tag{31}$$

  The corresponding optimality conditions are

$$f(x) = s_b - s_a \tag{32a}$$

$$0 \le s_b \perp (1-y) \ge 0 \tag{32b}$$

$$0 \le s_a \perp (y+1) \ge 0 \tag{32c}$$

  By substituting (32a) into (30) and by manipulating (32b) and (32c) to eliminate y we obtain the simplified model:

$$z = s_b + s_a \tag{33a}$$

$$f(x) = s_b - s_a \tag{33b}$$

$$0 \le s_b \perp s_a \ge 0 \tag{33c}$$

- The max operator $z = \max\{f(x), z_a\}$ can be rewritten as

$$z = f(x) + (z_a - f(x))y \tag{34}$$

$$y = arg\{\min_{\hat{y}}(f(x) - z_a)\hat{y} \quad \text{s.t.} \quad 0 \le \hat{y} \le 1\} \tag{35}$$

  The corresponding optimality conditions are

$$z = f(x) + (z_a - f(x))y \tag{36a}$$

$$f(x) - z_a = s_a - s_b \tag{36b}$$

$$0 \le s_b \perp (1-y) \ge 0 \tag{36c}$$

$$0 \le s_a \perp y \ge 0 \tag{36d}$$

These relations can be simplified by using (36b)–(36d) and substituting into (36a) to eliminate y, leading to:

$$z = f(x) + s_b \tag{37a}$$

$$f(x) - z_a = s_a - s_b \tag{37b}$$

$$0 \le s_b \perp s_a \ge 0 \tag{37c}$$

- The min operator $y = \min(f(x), y_b)$ can be treated in a similar way by defining the problem:

$$z = f(x) + (f(x) - z_b)y \tag{38}$$

$$y = arg\{\min_{\hat{y}}(f(x) - z_b)\hat{y} \quad \text{s.t.} \quad 0 \le \hat{y} \le 1\} \tag{39}$$

Applying the optimality conditions and simplifying leads to the following complementarity system:

$$z = f(x) + s_a \tag{40a}$$

$$z_b - f(x) = s_b - s_a \tag{40b}$$

$$0 \le s_b \perp s_a \ge 0 \tag{40c}$$

5.2. Flow reversal

Flow reversal can occur in many places in chemical processes including fuel headers and pipeline distribution networks. This is problematic for modeling since physical properties and other equations implicitly assume the sign of the flowrate to be positive. These situations may be modeled with the absolute value operator, using complementarities as in (33). The magnitude of the flowrate can then be used in the sign sensitive equations.

On the other hand, it may not be possible to implement these changes in some modeling environments, such as commercial process optimization software with directional streams and integrated thermo-physical models. In this case, each stream with the potential for flow reversal can be modeled as two parallel counter–current streams. The stream flowrates are then complemented, such that either one stream or the other has zero flow. Using mixers and splitters as appropriate, the two streams can then be integrated into any process flowsheet (Getting Started with ROMeo, 2004). While this method may obscure the original representation of the system, it remains equivalent.

5.3. Relief valves, check valves, compressor kick-back

Relief valves, check valves and compressor kick-back operation are all related phenomenon that can be modeled using complementarities in a manner similar to the functions used in Section 5.1.

- Check valves prevent fluid flows in reverse directions that may arise from changes in differential pressure. Assuming that the directional flow is a monotonic function of the differential pressure, we model flow through the check valve as:

$$\text{flow} = \max\{0, f(\Delta p)\}$$

and rewrite the max operator as the complementarity system given in (37).

- As shown in Fig. 3, compressor kickback, where recirculation is required when the compressor feed drops below a critical value, can be modeled in a similar manner. Following the stream notation in Fig. 3 we can write

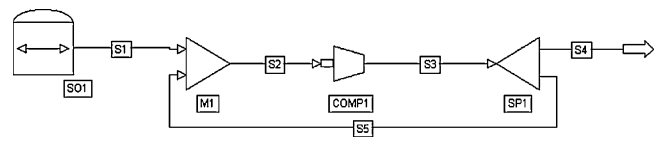

Fig. 3. Flowsheet example of compressor kick-back.

$$f_{S2} = \max(f_{S1}, f_{\text{crit}}), \quad f_{S5} = f_{\text{crit}} - f_{S1}$$

and rewrite the max operator using the complementarities in (37).

- Relief valves open and allow flow only when a pressure, p, or pressure differential, Δp, is larger than a predetermined value. Once open the flow is some function of the pressure, $f(p)$. This can be modeled as

$$\text{flow} = f(p)y \tag{41}$$

$$\min_{\hat{y}} \quad (p_{\max} - p)\hat{y} \tag{42a}$$

$$\text{s.t.} \quad 0 \le \hat{y} \le 1 \tag{42b}$$

The inner minimization problem sets $y = 1$ if $p > p_{\max}$ and sets $y = 0$ if $p < p_{\max}$. This zero–one behavior determines if the flowrate should be zero, when valve is closed, or determined by the expression $f(p)$, when the valve is open. The related complementarity conditions are

$$\begin{aligned} &\text{flow} = f(p)y \\ &(p_{\max} - p) = s_0 - s_1 \\ &0 \le y \perp s_0 \ge 0 \\ &0 \le (1-y) \perp s_1 \ge 0 \end{aligned}$$

5.4. Piecewise functions

Piecewise smooth functions are often encountered in physical property models, tiered pricing and table lookups. The composite function can be represented by the following inner minimization problem and associated equation (Raghunathan, 2004):

$$\min_{y} \quad \sum_{i=1}^{N} (x - a_i)(x - a_{i-1})y_i \tag{43a}$$

$$\text{s.t.} \quad \sum_{i=1}^{N} y_i = 1 \tag{43b}$$

$$y_i \ge 0 \tag{43c}$$

$$z = \sum_{i=1}^{N} f_i(x) y_i \tag{44}$$

where N is the number of piecewise segments, $f_i(x)$ is the function over the interval $x \in [a_{i-1}, a_i]$, and z represents the value of the piecewise function. This LP will set $y_i = 1$ and $y_{j \ne i} = 0$ when $x \in [a_{i-1}, a_i]$. The associated equation will then set $z = f_i(x)$, which is the function value on the interval. The NLP (43) can be rewritten as the following complementarity system:

$$\sum_{i=1}^{N} y_i = 1 \tag{45a}$$

$$(x - a_i)(x - a_{i-1}) - \gamma - s_i = 0 \tag{45b}$$

$$0 \le y_i \perp s_i \ge 0 \tag{45c}$$

In some cases, the function of interest may only be piecewise continuous and y_i can "cheat" by taking fractional values. For instance, cost per unit may increase stepwise over different ranges, and jumps in the costs may occur at a_i. A way around this problem would be to define $f_i(x)$ as the total cost (i.e., unit cost times quantity), which is continuous everywhere, but not differentiable at a_i.

5.5. PI controller saturation

PI controller saturation has been studied using complementarity formulations (Young, Baker, & Swartz, 2004). The PI control law takes the following form:

$$u(t) = K_c \left(e(t) + \frac{1}{\tau_I} \int_0^t e(t')\,\mathrm{d}t' \right) \tag{46}$$

where $u(t)$ is the control law output, $e(t)$ is the error in the measured variable, K_c is the controller gain and τ_I is the integral time constant. The controller output $v(t)$ is typically subject to upper and lower bounds, v_{up} and v_{lo}, i.e., $v(t) = \max(v_{lo}, \min(v_{up}, u(t)))$. All following variables are functions of time but have not been marked as such for notational convenience. The following inner minimization relaxes the controller output to take into account the saturation effects:

$$\begin{aligned} \min_{y_{up}, y_{lo}} \quad & (v_{up} - u(t))y_{up} + (u(t) - v_{lo})y_{lo} \\ \text{s.t.} \quad & 0 \le y_{lo}, y_{up} \le 1 \end{aligned} \tag{47}$$

with $v(t) = u(t) + (v_{up} - u(t))y_{up} + (v_{lo} - u(t))y_{lo}$. Applying the KKT conditions to (47) leads to

$$\begin{aligned} v(t) = u(t) + (v_{up} - u(t))y_{up} + (v_{lo} - u(t))y_{lo} \\ (v_{up} - u(t)) - s_{0,up} + s_{1,up} = 0 \\ (u(t) - v_{lo}) - s_{0,lo} + s_{1,lo} = 0 \\ 0 \le s_{0,up} \perp y_{up} \ge 0, \quad 0 \le s_{1,up} \perp (1 - y_{up}) \ge 0 \\ 0 \le s_{0,lo} \perp y_{lo} \ge 0, \quad 0 \le s_{1,lo} \perp (1 - y_{lo}) \ge 0 \end{aligned} \tag{48}$$

Eliminating the variables y_{up} and y_{lo} and applying the complementarity conditions leads to the following simplification:

$$\begin{aligned} v(t) = u(t) + s_{1,lo} - s_{1,up} \\ (v_{up} - u(t)) - s_{0,up} + s_{1,up} = 0 \\ (u(t) - v_{lo}) - s_{0,lo} + s_{1,lo} = 0 \\ 0 \le s_{1,up} \perp s_{0,up} \ge 0, \quad 0 \le s_{0,lo} \perp s_{1,lo} \ge 0 \end{aligned} \tag{49}$$

5.6. Disappearance of phases

The disappearance of phases in vapor liquid equilibrium systems was considered in Gopal and Biegler (1999). Here the equilibrium between two phases is relaxed by an additional variable β and defined by $y_j = \beta K_j x_j$. For a flash vessel with feed F, the complementarity system can be derived from the following inner minimization problem:

$$\min_{L,V} \quad (1 - \beta)(L - V) \tag{50a}$$

$$\text{s.t.} \quad L + V = F \tag{50b}$$

$$L, V \ge 0 \tag{50c}$$

leading to the following complementarity system:

$$y_j = \beta K_j x_j \tag{51a}$$

$$\beta = 1 - s^l + s^v \tag{51b}$$

$$0 \le L \perp s^l \ge 0 \tag{51c}$$

$$0 \le V \perp s^v \ge 0 \tag{51d}$$

In this manner, phase existence can be determined within the context of an optimization problem. If either liquid or vapor is absent, the corresponding slack variable may become positive. This in turn relaxes the calculation of the phase equilibrium, which is appropriate since only one phase is present. As seen in Gopal and Biegler (1999), Lang and Biegler (2002), Raghunathan and Biegler (2003), and Raghunathan et al. (2004), this formulation can also be extended to model distillation columns, and, as shown in Gopal and Biegler (1999), the conditions are also equivalent to minimization of the Gibbs free energy.

6. Case studies

From the results of Sections 3 and 4, we find that the penalty formulation performs well, particularly with an active set solver such as CONOPT. With this examination of NLP reformulations for MPCC as well as a systematic strategy for developing complementarity formulations, we now consider two large-scale case studies that illustrate both of these concepts. The first considers discrete decisions in dynamic systems with the reformulation of the *signum* function, while the second considers the disappearance of phases in optimization models for distillation.

6.1. Dynamic optimization with direct transcription

The goal of this case study is to demonstrate the NLP formulations in Section 3 on an example with arbitrarily many complementarity conditions. The example problem minimizes an arbitrary objective function $\phi = (x_{end} - 5/3)^2 + \int_{t_0}^{t_{end}} x^2 \mathrm{d}t$ subject to a differential inclusion $\dot{x} \in \mathrm{sgn}(x) + 2$ with initial condition $x_0 = -2$, where the derivative can be defined by a complementarity system (Stewart and Anitescu, 2005). While we do not consider limiting properties of the discretization, our numerical results will focus on how the solution time varies as a function of the problem size (controlled by the discretization strategy).

The differential inclusion can be reformulated as the following optimal control problem:

$$\min \quad (x_{end} - 5/3)^2 + \int_{t_0}^{t_{end}} x^2\,\mathrm{d}t \tag{52a}$$

$$\text{s.t.} \quad \dot{x} = u + 2 \tag{52b}$$

$$x = s^+ - s^- \tag{52c}$$

$$0 \le 1 - u \perp s^+ \ge 0 \tag{52d}$$

$$0 \le u + 1 \perp s^- \ge 0 \tag{52e}$$

Applying the implicit Euler's method with a fixed step size, the discretized problem becomes

$$\min \quad (x_{end} - 5/3)^2 + \sum_{i=1}^{NFE} h x_i^2 \tag{53a}$$

$$\text{s.t.} \quad \dot{x}_i = u_i + 2 \quad i = 1, \ldots, \text{NFE} \tag{53b}$$

$$x_i = x_{i-1} + h\dot{x}_i \quad i = 1, \ldots, \text{NFE} \tag{53c}$$

$$x_i = s_i^+ - s_i^- \quad i = 1, \ldots, \text{NFE} \tag{53d}$$

$$0 \le 1 - u_i \perp s_i^+ \ge 0 \quad i = 1, \ldots, \text{NFE} \tag{53e}$$

$$0 \le u_i + 1 \perp s_i^- \ge 0 \quad i = 1, \ldots, \text{NFE} \tag{53f}$$

The resulting discretized problem can be scaled up to be arbitrarily large by increasing the number of finite elements *NFE*, with a discrete decision in each element. This system was modeled in GAMS and solved via automatic reformulation using the NLPEC package.

Table 2
Solution times (Pentium 4, 1.8 GHz, 992 MB RAM) for different solution strategies

NFE	Objective function	SBB CPUs	CONOPT-PF		IPOPT-C	
			CPUs	Iterations	CPUs	Iterations
10	1.4738	0.242	0.047	15	0.110	17
100	1.7536	50.266	0.234	78	1.250	41
1000	1.7864	410.195	9.453	680	28.406	78
2000	1.7888	> 6000	35.359	1340	14.062	25
3000	1.7894	–	112.094	2020	106.188	84
4000	1.7892	–	211.969	2679	84.875	56
5000	1.7895	–	340.922	3342	199.391	87
6000	1.7898	–	468.891	3998	320.140	115
7000	1.7896	–	646.953	4655	457.984	141
8000	1.7898	–	836.891	5310	364.937	98

Additionally, the GAMS model was converted to an AMPL model and the problems were solved using IPOPT-C.

This dynamic optimization problem can also be modeled as a mixed integer quadratic program (MIQP). Using concepts from generalized disjunctive programming (GDP) one can introduce binary variables, b_i, and replace (53d)–(53f) by the following constraints:

$$x_L(b_i - 1) \le x_i \le x_U \; b_i \qquad i = 1, \ldots, \text{NFE} \tag{54a}$$

$$u_i = 2b_i - 1 \qquad i = 1, \ldots, \text{NFE} \tag{54b}$$

$$b_i \in \{0, 1\} \qquad i = 1, \ldots, \text{NFE} \tag{54c}$$

However, finding a solution to this MIQP is NP-Hard and with finer discretizations the problem can become expensive. In contrast, solving (53) through an NLP formulation generally leads to only polynomial complexity. On the other hand, the resulting NLP model is nonconvex and global optimality cannot be ensured, although on this example a comparison with MIQP solvers showed that global solutions were always obtained by the MPCC solver.

The discretized MPCC was solved using the penalty reformulation (PF(ρ) with CONOPT using $\rho = 1000$). The results are shown in Table 2. It can be seen that the number of iterations increases almost linearly with the number of finite elements and this behavior is typical for active set strategies. In addition, the computational time per iteration increases linearly with the number of finite elements, as the majority of CPU time is expected to be spent in the sparse linear solver. In this case, sparse linear solvers increase approximately linearly in computation time with problem size. Consequently, the total computational time for CONOPT-PF grows approximately quadratically with problem size.

IPOPT-C was also used to solve this problem and the results are shown in Table 2. It can be seen that the number of iterations increases only weakly as a function of problem size, with no clear trend. In fact, the iteration count seems to saturate with problem size. In addition, the computational time per iteration also increases linearly with the number of finite elements. This is expected since the largest computational cost of IPOPT is due to the linear solver. Here, this solver cost increases linearly with problem size. As a result, the total computational time increase is only slightly superlinear with problem size. Moreover, it is also notable that while CONOPT is faster for the smaller problems, IPOPT-C becomes faster for the larger problems. This is consistent with expectations. CONOPT must also deal with the combinatorial complexity of identifying the active set at every iteration, whereas the barrier method IPOPT-C identifies the active set only at the solution.

Finally, the SBB solver, a simple branch and bound solver in GAMS, was applied to the above MIQP formulation of this problem. From the results in Table 2, we can observe the NP-hard aspect of solving the MIQP with this approach. Solving the problem with 1000 finite elements requires over 400 CPUs and the problem with 2000 finite elements cannot be solved within 6000 CPUs.

6.2. Distillation optimization

As seen in Section 5.6, phase behavior of a vapor liquid system is determined by the minimization of the Gibbs free energy and can be modeled through complementarity constraints. These conditions allow phase disappearance to be modeled in distillation systems for optimization of both steady state and dynamic tray columns (Lang & Biegler, 2002; Raghunathan & Biegler, 2003; Raghunathan et al., 2004). In these studies smoothing methods and regularization methods were applied for distillation optimization, with the reflux ratio, feedtray location and tray number as decision variables. In this section, we revisit these steady state distillation optimizations for the sake of a performance comparison, particularly with the penalty formulation, PF(ρ).

Here, we define distillation column models that consist of mass balance, equilibrium, summation, and heat balance (MESH) equations that are modified to optimize the feed tray location and total tray count. As shown in Fig. 4, streams for the feed and the reflux are fed to all trays as dictated by two discretized Gaussian distribution functions. As developed and described in Lang and Biegler (2002), both distribution functions are defined by a mean and standard deviation. For the feed and reflux distributions, their means represent the feedtray location and the total number of trays, respectively, and these are used as continuous decision variables in the distillation optimization. We also choose a standard deviation of 0.5, so that distribution flow to trays away from the mean tapers off smoothly to negligible values. Moreover, the grayed area in Fig. 4

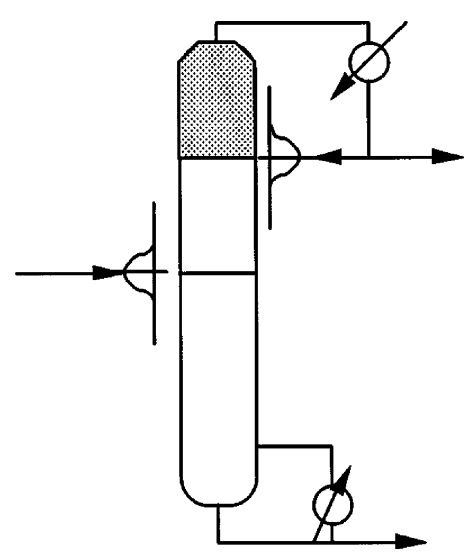

Fig. 4. Diagram of distillation column showing that the feed and reflux flows are distributed across multiple trays according to the DDF. The grayed section of the column is above the primary reflux location and has negligible liquid flows.

consists only of vapor traffic, and consequently, each tray model must include complementarities that allow for disappearance of the liquid phase. As described in Section 5.6, phase equilibrium is relaxed by the complementarity constraints:

$$y_{ij} = \beta_i K_{ij} x_{ij} \tag{55a}$$

$$\beta_i = 1 - s_i^l + s_i^v \tag{55b}$$

$$0 \le L_i \perp s_i^l \ge 0 \tag{55c}$$

$$0 \le V_i \perp s_i^v \ge 0 \tag{55d}$$

where $i = \{1, 2, \ldots, N_{\max}\}$ and $N_{\max}$ is the maximum number of equilibrium stages, j is the component index, L_i and V_i are the liquid and vapor flow rate on tray i, and x and y are the respective mole fractions. Trays are numbered from the bottom to the top of the column.

The resulting complementarity distillation model is an alternative nonlinear program for distillation design and optimization. As shown in Lang and Biegler (2002) and Raghunathan et al. (2004), it determines the optimal number of trays, reflux ratio and feedtray location for detailed column models. As case studies we consider two systems drawn from Lang and Biegler (2002). The first system is a benzene/toluene separation and the second system is an argon column from an air separation unit. Both models use ideal thermodynamics, are modeled in GAMS and solved with CONOPT. We use CONOPT as the solver in these case studies because equations may become dependent during the course of the optimization; CONOPT can better handle the dependencies with its active set strategy than IPOPT. Three MPCC formulations were considered for these cases:

- Penalty formulation, PF(ρ) with $\rho = 1000$ chosen as the penalty weight for the benzene/toluene separation and $\rho = 10^6$ for the argon column.
- Relaxed formulation, Reg(ϵ). This strategy was solved in two NLP stages with $\epsilon = 10^{-6}$ followed by $\epsilon = 10^{-12}$. This formulation was also applied to distillation optimization in Raghunathan et al. (2004).
- NCP formulation using the Fischer–Burmeister function (22). This approach is equivalent to RegEq(ϵ) and was solved in three NLP stages with $\epsilon = 10^{-4}$ followed by $\epsilon = 10^{-8}$ and $\epsilon = 10^{-12}$. This strategy was applied to distillation optimization in Lang and Biegler (2002).

6.2.1. Benzene–toluene column optimization

This binary column has a maximum of 25 trays, its feed is 100 mol/s of a 70%/30% mixture of benzene/toluene and distillate flow is specified to be 50% of the feed. The reflux ratio is allowed to vary between 1 and 20, the feedtray location varies between 2 and 20 and the total tray number varies between 3 and 25. The objective function for the benzene–toluene separation minimizes:

$$\text{objective} = \text{wt}\, D x_{D,\text{Toluene}} + \text{wr}\, r + \text{wn}\, N_t \tag{56}$$

where N_t is the number of trays, r is the reflux ratio, D is distillate flow, $x_{D,\text{Toluene}}$ is the toluene mole fraction and wt, wr, and wn are the weighting parameters for each term; these weights allow the optimization to trade off product purity, energy cost and capital cost. The benzene–toluene optimizations were initialized with 21 trays, a feedtray location at the seventh tray and a reflux ratio at 2.2. Temperature and mole fraction profiles were initialized with linear interpolations based on the top and bottom product properties. The resulting GAMS models consisted of 353 equations and 359 variables for the Reg(ϵ) and NCP formulations, and 305 equations and 361 variables for the PF(ρ) formulation. The following cases were considered:

Table 3
Distillation optimization: comparison of MPCC formulations

Cases	1	2	3	Argon
iter-PF(ρ)	356	395	354	1182
iter-Reg(ϵ)	538	348	370	10269
iter-NCP	179	119	254	328
CPUs-PF(ρ)	7.188	7.969	4.875	69.281
CPUs-Reg(ϵ)	14.594	7.969	7.125	706.406
CPUs-NCP	7.906	3.922	9.188	20.781
obj-PF(ρ)	9.4723	6.8103	3.0053	77.248
obj-Reg(ϵ)	9.5202 [a]	6.8103 [a]	2.9048 [a]	82.491 [a]
obj-NCP	11.5904 [a]	9.7288 [a]	2.9332 [a]	88.406 [a]
reflux-PF(ρ)	1.619	4.969	1.485	32.248
reflux-Reg(ϵ)	1.811	4.969	1.431	33.491
reflux-NCP	1.683	2.881	1.359	38.406
N_t-PF(ρ)	6.524	5.528	8.844	45
N_t-Reg(ϵ)	6.645	5.528	9.794	49
N_t-NCP	9.437	9.302	9.721	50

All computations were performed on a Pentium 4 with 1.8 GHz and 992 MB RAM.
[a] CONOPT terminated at feasible point with negligible improvement of the objective function, but without satisfying KKT tolerance.

- Case 1 (wt = 1, wr = 1, wn = 1): This represents the base case with equal weights on toluene in distillate, reflux ratio and tray count. As seen in the results in Table 3, the optimal solution has an objective function value of 9.4723 with intermediate values of r and N_t. This is found quickly by the PF(ρ) formulation. On the other hand, the Reg(ϵ) formulation terminates close to this solution, while the NCP formulation terminates early with poor progress.
- Case 2 (wt = 1, wr = 0.1, wn = 1): In this case, less emphasis is given to energy cost. As seen in the results in Table 3, the optimal solution now has a lower objective function value of 6.8103 along with a higher value of r and lower value of N_t. This is found quickly by both PF(ρ) and Reg(ϵ), although only the former satisfies the convergence tolerance. On the other hand, the NCP formulation again terminates early with poor progress.
- Case 3 (wt = 1, wr = 1, wn = 0.1): In contrast to Case 2, less emphasis is now given to capital cost. As seen in the results in Table 3, the optimal solution now has an objective function value of 2.9048 with lower values of r and higher values of N_t. This is found quickly by the Reg(ϵ) formulation. Although Reg(ϵ) does not satisfy the convergence tolerance, the optimum could also be verified by PF(ρ). On the other hand, PF(ρ) quickly converges to a slightly different solution, which it identifies as a local optimum, while the NCP formulation requires more time to terminate with poor progress.

6.2.2. Argon column optimization

This argon separation column has a maximum of 63 trays, its feed is 6546.54 lbmol/h of a 0.005%/9.753%/90.24% mixture of nitrogen/argon/oxygen and distillate flow is specified to be 202.4576 lbmol/h with less than 1 mol% oxygen. The reflux ratio is allowed to vary between 20 and 100, the feedtray location varies between 1 and 10 and the total tray number varies between 31 and 63. The objective function for the argon problem minimizes: objective $= r + N_t$.

The argon column optimizations were initialized with 50 trays, a feedtray location at the first tray and a reflux ratio at 32.8. Temperature and mole fraction profiles were initialized with linear interpolations based on the top and bottom product properties. The resulting GAMS models consist of 1260 equations and 1267 variables for the Reg(ϵ) and NCP formulations, and 1136 equations and 1269 variables for the PF(ρ) formulation.

As seen in the results in Table 3, the optimal solution has an objective function value of 77.248. This was found relatively quickly

by the PF(ρ) formulation. On the other hand, the Reg(ϵ) formulation terminates close to this solution but requires an order of magnitude more effort. Again, the NCP formulation terminates early with poor progress.

Along with the numerical study in Section 4, these four cases demonstrate that optimization of detailed distillation column models can be performed efficiently with MPCC formulations. In particular, it can be seen that the penalty formulation (PF(ρ)) represents a significant improvement over the NCP formulation both in terms of iterations and CPU seconds; PF(ρ) offers advantages over the Reg(ϵ) formulation as well.

7. Conclusions

With the development and widespread use of large-scale nonlinear programming tools for process optimization, there has been an associated application of complementarity formulations to represent discrete decisions. This study investigates both the formulation and solution strategies for the MPCCs. Because MPCCs have dependent constraints at all feasible points along with unbounded multipliers, special care is needed in the formulation of associated optimization problems along with a reliable solution algorithm. In this study, we investigate and summarize a number of regularization and penalty MPCC formulations and find that the penalty formulation is particularly advantageous, especially when coupled with an active set NLP solver. This conclusion is observed in a numerical comparison with a library of MPCC test problems.

Because all MPCCs can be formulated as underlying MPEC problems that result from nested optimization formulations, a necessary condition for well-behaved MPCCs is that the inner optimization problem be convex with solutions that are not restricted by upper level constraints. This condition is used to motivate a systematic strategy for the formulation of complementarity constraints. This strategy is applied to a number of widely used examples in process engineering, including flow reversals, relief valves, compressor kickbacks, controller saturation and disappearance of phases in equilibrium stages.

This combination of well-posed formulations and MPCC solutions strategies is demonstrated on two large-scale case studies with as many as 8000 discrete decisions. First, in the discretized dynamic system with switches, both the penalty and relaxed formulations are very effective with active set and barrier solvers, respectively. Second, in dealing with disappearing phases in distillation optimization models, the penalty formulation shows significant performance improvements over the NCP formulation in a previous study. From these results, we find that well-posed complementarities coupled with NLP problems based on penalty formulations are efficient and effective strategies for the solution of MPCCs in process engineering.

Acknowledgements

Funding from the National Science Foundation under grants #CTS-0438279 and CTS-0457379 is gratefully appreciated. The authors also wish to thank Dr. Steven Dirkse for fruitful discussions and help with the NLPEC package.

References

Anitescu, M., Tseng, P., & Wright, S. J. (2007). Elastic-mode algorithms for mathematical programs with equilibrium constraints: Global convergence and stationarity properties. *Mathematical Programing*, *110*, 337–371.

Burke, J. V., & Han, S.-P. (1989). A robust sequential quadratic programming method. *Mathematical Programing*, *43*, 277–303.

Chen, X., & Fukushima, M. (2004). A smoothing method for a mathematical program with P-matrix linear complementarity constraints. *Computation Optimization and Applications*, *27*, 223–246.

Ferris, M. C., & Pang, J. S. (Eds.). (1997). *Complementarity and Variational Problems: State of the Art*. Philadelphia, Pennsylvania: SIAM Publications.

Ferris, M. C. (2005). *Complementarity Problems and Applications*, presented at SIAM Conference on Optimization, Stockholm.

Fletcher, R., & Leyffer, S. (2004). Solving mathematical programs with complementarity constraints as nonlinear programs. *Optimization Methods and Software*, *19*(1), 15–40.

Gallun, S. E., Luecke, R. H., Scott, D. E., & Morshedi, A. M. (1992). Use open equations for better models. *Hydrocarbon Processing*, 78–90.

GAMS. (2004). *The Solver Manuals*. Washington, DC: GAMS Development Corporation.

Getting Started with ROMeo. (2004). SimSci-Esscor.

Gopal, V., & Biegler, L. T. (1999). Smoothing methods for complementarity problems in process engineering. *Journal of AIChE*, *45*(7), 1535–1547.

Hu, X. M., & Ralph, D. (2004). Convergence of a penalty method of mathematical programming with complementarity constraints. *Journal of Optimization Theory and Applications*, *123*(2), 365–390.

Heemels, M. (1999). PhD Thesis, Technische Universiteit Eindoven.

Kameswaran, S., Staus, G., & Biegler, L. T. (2005). Parameter estimation of core flood and reservoir models. *Computers and Chemical Engineering*, *29*(8), 1787–1800.

Lang, Y.-D., & Biegler, L. T. (2002). Distributed stream method for tray optimization. *AIChE Journal*, *48*(3), 582–595.

Leyffer, S., Lopez-Calva, G., & Nocedal, J. (2006). Interior methods for mathematical programs with complementarity constraints. *SIAM Journal of Optimization*, *17*(1), 52–77.

Lopez-Calva, G. (2005). Regularization via Exact Penalty Methods. In *4th International Conference on Complementarity Problems, August 10th*.

Luo, Z. Q., Pang, J. S., & Ralph, D. (1996). *Mathematical Programs with Equilibrium Constraints*. Cambridge: Cambridge University Press.

Mittelmann, H. D., & Pruessner, A. (2006). A server for automated performance analysis of benchmarking data. *Optimization Methods and Software*, *21*(1), 105–120.

MPEC Library, http://www.gamsworld.eu/mpec/mpeclib.htm.

Raghunathan, A., & Biegler, L. T. (2003). MPEC formulations and algorithms in process engineering. *Computers and Chemical Engineering*, *27*, 1381–1392.

Raghunathan, A. (2004). PhD Thesis, Department of Chemical Engineering, Carnegie Mellon University.

Raghunathan, A. U., & Biegler, L. T. (2005). Interior point methods for mathematical programs with complementarity constraints (MPCCs). *SIAM Journal of Optimization*, *15*(3), 720–750.

Raghunathan, A., Diaz, M. S., & Biegler, L. T. (2004). An MPEC formulation for dynamic optimization of distillation operation. *Computers and Chemical Engineering*, *28/10*, 2037–2052.

Raghunathan, A. U., Perez-Correa J. R., Agosin E., & Biegler L. T. (2006). Parameter estimation in metabolic flux balance models for batch fermentation – formulations and solution using differential variational inequalities. *Annals of Operations Research, 148*, 251–270.

Ralph, D., & Wright, S. J. (2004). Some properties of regularization and penalization schemes for MPECs. *Optimization Methods and Software*, *19*(5), 527–556.

Scheel, H., & Scholtes, S. (2000). Mathematical programs with complementarity constraints: Stationarity, optimality, and sensitivity. *Mathematics of Operations Research*, *25*(1)

Scholtes, S., & Stöhr, M. (2001). How stringent is the linear independence assumption for mathematical programs with complementarity constraints? *Mathematics of Operations Research*, *26*(4), 851–863.

Stein, O., Oldenburg, J., & Marquardt, W. (2004). Continuous reformulations of discrete–continuous optimization problems. *Computers & Chemical Engineering*, *28*(10), 1951–1966.

Stewart, D., & Anitescu, M. (June 2005). *Optimal Control of Systems with Discontinuous Differential Equations*. Preprint number: ANL/MCS-P1258-0605, Mathematics and Computer Science Division, Argonne National Laboratory.

Young, J. C. C., Baker, R., & Swartz, C. L. E. (2004). Input saturation effects in optimizing control: Inclusion within a simultaneous optimization framework. *Computers and Chemical Engineering*, *28*, 1347–1360.